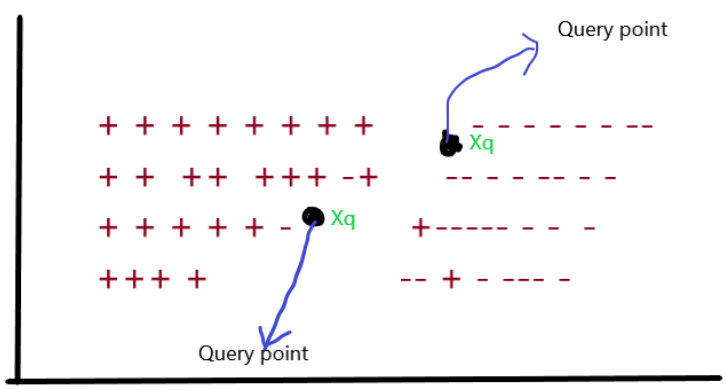
How to choose the right Distance Metric

K- Nearest Neighbours

This is a supervised machine learning algorithm, which is generally used to solve classification problems but sometimes it was also used in regression problems too. The main aim of KNN is to find the nearest neighbours of our query point. This algorithm believes that similar things are in close proximity, in other words, we can say that suppose X is +ve in a group of points so there is a high chance that the point nearer to X is also +ve. Let us talk about the concept of nearest neighbour for better understanding.

Nearest Neighbour:

let's take the simplest case of binary classification, suppose we have a group of +ve and -ve points in the dataset D such that the Xi_s belongs to



the R-dim. data points and Y_i are labels (+ve and -ve).

From the above image, you can conclude that there are several data points in 2 dim. Having the specific label, they are classified according to the +ve and -ve labels. If you noticed in the image there is one Query point referred to as Xq which has an unknown label. The surrounding points

of Xq we considered as neighbours of Xq and the points which are close to the Xq are nearest neighbours .

So how can we conclude that this point is nearest or not? It's by finding the distance b/w the points. So, here's the distance measures come existence.

Distance measures

We generally say that we will use distance to find the nearest neighbours of any query point Xq, but we still don't know how mathematically distance is measured between Xq and other nearest points? for further finding distance, we can't conclude that this is nearest or not.

In a theoretical manner, we can say that a distance measure is an objective score that summarizes the difference between two objects in a specific domain. There are several types of distance measures techniques but we only use some of them and they are listed below:

1. Euclidean distance

2. Manhattan distance

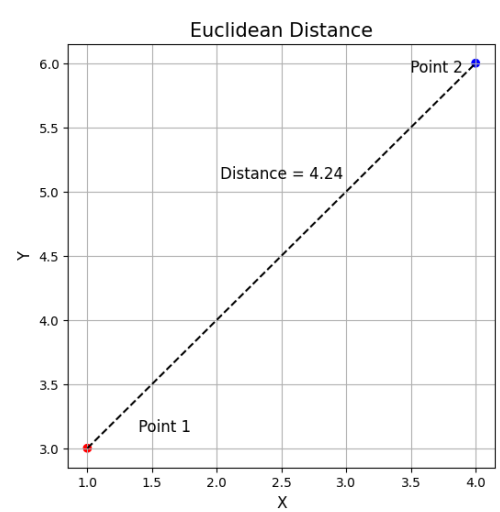
3. Minkowski distance

4. Hamming distance

Euclidean Distance

Used commonly for continuous data, it's the straight-line distance between two points in Euclidean space.

$$[d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}]$$



In this equation:

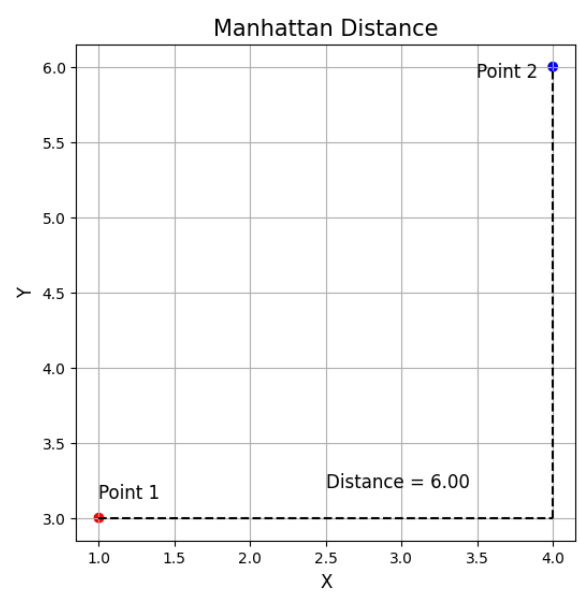
- x_i and y_i are the coordinates of points x and y in the i-th dimension, respectively.
- The term $(x_i - y_i)^2$ computes the squared difference between the coordinates of x and y in each dimension.
- The summation $\sum$ adds up these squared differences across all dimensions.
- The square root is applied to the sum of squared differences, yielding the final distance.

In the image, this would be

$$
\begin{aligned}
d &= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \\
&= \sqrt{(4-1)^2 + (6-3)^2} \\
&= \sqrt{9+9} \\
&= \sqrt{18} \\
&\approx 4.24
\end{aligned}
$$

Manhattan Distance

Also known as the city block distance, it is the sum of the absolute differences of their Cartesian coordinates. Unlike the straight-line distance measured by Euclidean distance,



Manhattan distance calculates the distance traveled along axes at right angles. It's preferred for categorical data.

$$[d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} |x_i - y_i|]$$

The term $|x_i - y_i|$ calculates the absolute difference between the coordinates of x and y in each dimension.

- The summation $\sum$ aggregates these absolute differences across all dimensions.

Following the example above this would be:

$$\begin{aligned} d &= |x_2 - x_1| + |y_2 - y_1| \\ &= |4 - 1| + |6 - 3| \\ &= 3 + 3 \\ &= 6 \end{aligned}$$

Minkowski Distance

It's a generalization of both Euclidean and Manhattan distances. It introduces a parameter *p* that allows different distance metrics to be calculated. The Minkowski distance includes both the Euclidean distance and the Manhattan distance as special cases when *p*=2 and *p*=1, respectively.

$$[d(\mathbf{x}, \mathbf{y}) = (\sum_{i=1}^{n} |x_i - y_i|^p)^{\frac{1}{p}}]$$

Here:

- $|x_i - y_i|$ calculates the absolute difference between the coordinates of x and y in the *i*-th dimension.

- *p* is a positive integer that determines the order of the Minkowski distance. When *p* changes, the nature of the distance measurement changes as well.
- The summation $\sum$ aggregates these absolute differences, raised to the power of *p*, across all dimensions.
- Finally, the *p*-th root of the sum gives the Minkowski distance.

Note: Minkowski distance between two vectors or points is the Lp norm of two vector.

Example:So, here we will take the same example as we take in the euclidean distance measures.

$X1(x_1, y_1) = X1(3, 4)$

$X2(x_2, y_2) = X2(4, 7)$

and we take the value of P = 4

distance = $((x_2-x_1)^4 + (y_2-y_1)^4)^{1/4}$

distance = $((4-3)^4 + (7-4)^4)^{1/4}$

distance = $((1)4 + (3)^4)^{1/4}$

distance = $(1+81)^{1/4}$

distance = $(82)^{1/4}$

distance = approx(2)

Hamming distance

This distance measurement technique is more different from that technique we have learned above, The hamming distance is mostly used in text processing or having the boolean vector. Boolean vector means the data is in the form of binary digits 0 and 1. So, the hamming distance

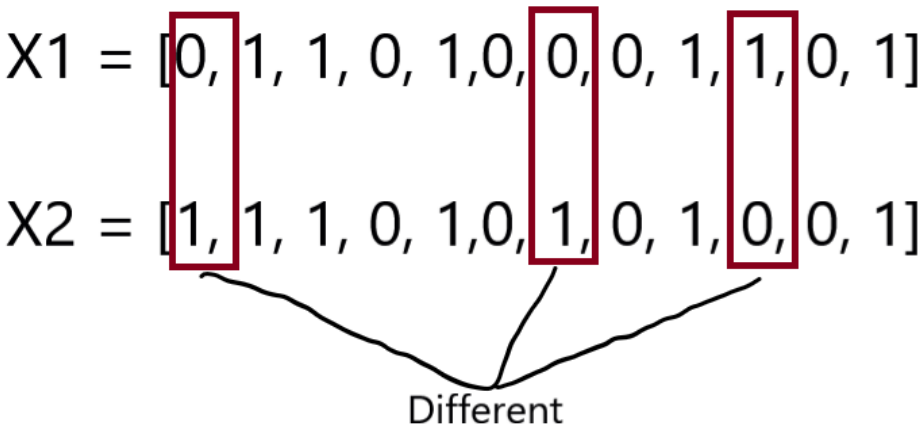
calculates the distance between two binary vectors, also referred to as binary strings.

Suppose we have two points X1 and X2 and both of them are boolean vectors, represented as:

X1 = [0,0,0,1,0,1,1,0,1,1,1,0,0,0,1]

X2 = [0,1,0,1,0,1,0,0,1,0,1,0,1,0,1]

So more simply the hamming distance of (X1, X2) is the location where the binary vectors or numbers are different. let's simplify for you;



As you see in the above image that there are 3 locations where the binary numbers are different or we can say that the one respective number doesn't match with the other one respectively.

So, we can say that the hamming distance of these two boolean vectors is 3. let's see more mathematically.

Hamming distance(X1, X2) = 3 …… {place where binary vectors are differ}

So this is the fundamental knowledge of the distance measures when we use KNN and other algorithms.

Example:

SO, this distance measure is different from the above 3, Suppose we have a problem that finds the number of locations where the string is different?

string1 = a n d v j s k e

string2 = a n r v j a k c

So you can notice that at 3 locations the string is different.

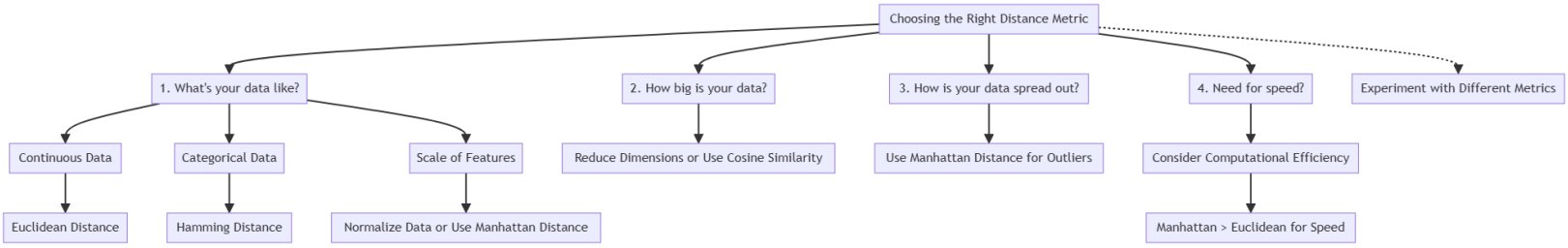
Choosing the right distance metric is also a critical step in optimizing the KNN for specific datasets and problem domains. Using an analogy, it's like choosing the right glasses to see the data clearly: the better the fit, the clearer you'll see your 'k' nearest neighbors and the better your predictions will be. To understand what's the best distance to use, you should ask yourself the following questions:

1. What's your data like? Continuous vs. Categorical: If your data is all about numbers and measurements (continuous data), Euclidean distance is your go-to, because it measures straight lines between points. For data that's more about categories (like types of fruit, where "apple" and "orange" aren't on a scale), Hamming distance, which checks if features match, makes more sense.

Scale of Features: Look out for different scales in your dataset. If you don't adjust for this, your distances could be thrown off, making some features louder than others. Normalize your data or switch to Manhattan distance, which isn't as thrown off by different scales.

2. How big is your data? When your dataset is really wide (lots of features), traditional ideas of closeness get wonky, and everything starts to seem far apart. Here, reducing dimensions or picking metrics suited for the big stage, like cosine similarity for text, can keep things in perspective.

3. How is your data spread out? The way your data is distributed matters. If outliers are a big deal in your dataset, Manhattan distance might be your ally since it doesn't get as shaken up by extreme values compared to Euclidean distance.

4. Need for speed? Some distance metrics are computationally more intensive than others. Metrics like Manhattan distance can be computationally more efficient than Euclidean distance in certain implementations since it lacks the square root operation.

Lastly, don't marry the first metric you meet. Play the field, try different metrics, and see which one makes your model the happiest through cross-validation.